

Báo Cáo Milestone 1: Data Acquisition

Môn học: SEG301 - Search Engines & Information Retrieval

Dự án: Birds Search Engine - Social Listening

Nhóm: Phan Minh Tài · Nguyễn Châu Thành Sơn · Trần Gia Phúc

Repository: github.com/SarenFan/Birds-search-engine

1. Tổng quan

1.1. Chủ đề dự án

Dự án thuộc **Topic 4 - Social Listening** (Lắng nghe Mạng xã hội), thu thập và xử lý dữ liệu từ diễn đàn Voz Forum — một trong những cộng đồng trực tuyến lớn nhất Việt Nam với hàng triệu bài viết đa dạng chủ đề.

1.2. Định nghĩa Document hợp lệ

Để phù hợp với bài toán Social Listening, nhóm định nghĩa một **Document hợp lệ** phải thỏa mãn:

1. Là một bài viết (Post) hoặc bình luận (Comment) trên diễn đàn Voz.
2. Độ dài tối thiểu: **50 từ** (sau khi đã tách từ và loại nhiễu).
3. Ngôn ngữ: Tiếng Việt (có hỗ trợ từ khóa tiếng Anh chuyên ngành như "laptop", "game", "bitcoin"...).
4. Nội dung không trùng lặp (unique content hash).

Các comment ngắn (như "hóng", "bóc tem", "hay quá", "+1") bị loại bỏ vì không mang giá trị thông tin (Information Value) cho Search Engine.

1.3. Mục tiêu Milestone 1

- Thu thập tối thiểu **1,000,000 documents** từ Voz Forum
- Làm sạch dữ liệu: loại HTML, xử lý teencode, tách từ tiếng Việt

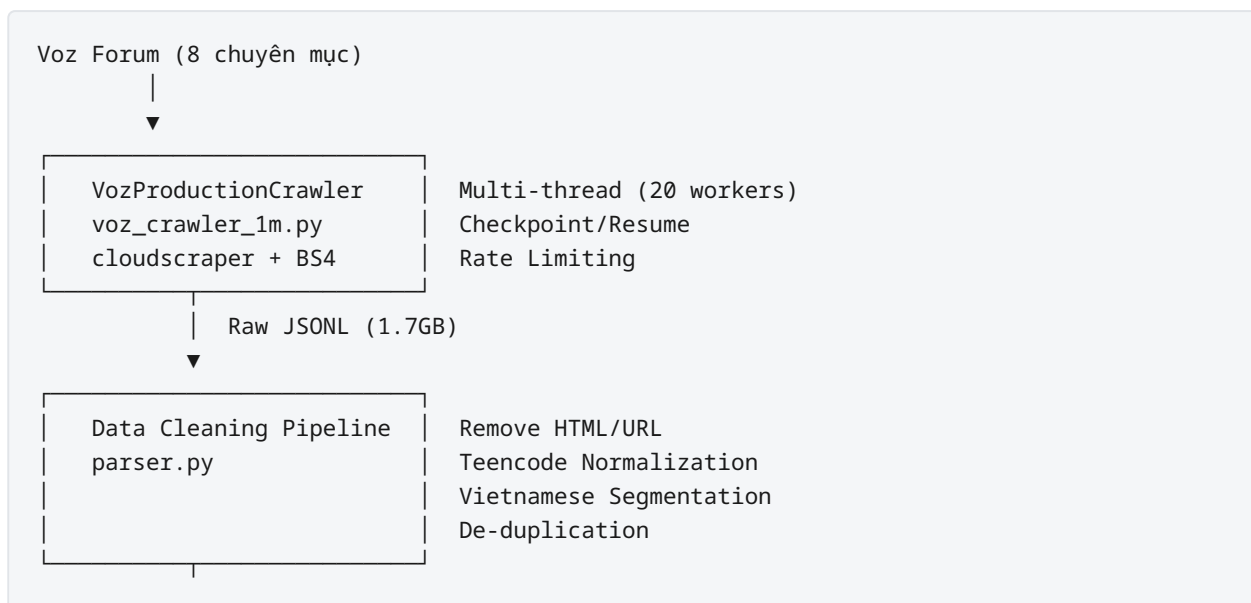
- Loại bỏ trùng lặp (De-duplication)
- Lưu trữ dạng JSONL có cấu trúc
- Báo cáo thống kê dữ liệu

1.4. Kết quả đạt được

Chỉ số	Giá trị
Tổng documents đầu vào (raw)	1,008,664
Duplicates loại bỏ	1,999
Tổng documents sau cleaning	1,000,083
Tổng số từ (tokens)	82,466,288
Kích thước từ vựng	822,837 từ duy nhất
Số lượng thread	100,030
Số lượng tác giả	37,629
Khoảng thời gian dữ liệu	03/2020 – 01/2026

2. Kiến trúc hệ thống

2.1. Pipeline tổng quan



Cleaned JSONL (2.8GB)

Statistics Generator
data_statistics.py

Word frequency
Distribution analysis
Author/Thread stats

2.2. Cấu trúc thư mục

```
Birds-search-engine/
├── src/
│   ├── crawler/
│   │   ├── voz_crawler_1m.py      # Crawler chính
│   │   ├── parser.py             # Pipeline làm sạch dữ liệu
│   │   └── statistics_report.py   # Báo cáo thống kê
│   ├── data_sample/
│   │   └── voz_cleaned_sample.jsonl # 500 docs mẫu
│   ├── docs/
│   │   └── Milestone1_Report.pdf
│   └── requirements.txt
```

3. Crawler - Thu thập dữ liệu

3.1. Nguồn dữ liệu

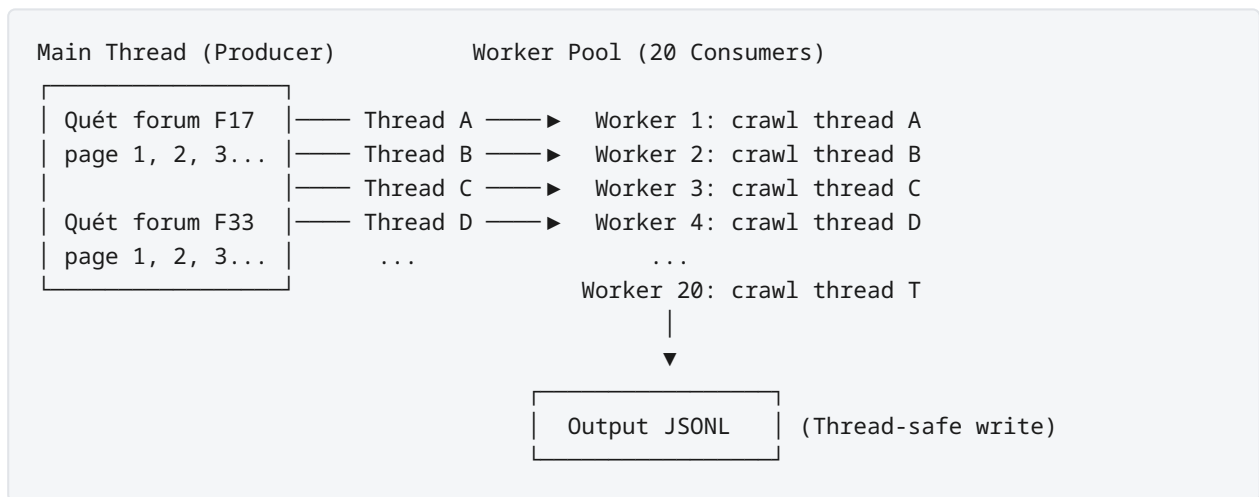
Crawler thu thập từ **8 chuyên mục** trên Voz Forum, luân phiên để đa dạng hóa nội dung:

```
# voz_crawler_1m.py (dòng 162-171)
FORUMS = {
    'f17': '/f/chuyen-tro-linh-tinh.17/', # Chuyện trò linh tinh
    'f33': '/f/the-gioi-game.33/',       # Thế giới game
    'f10': '/f/may-tinh.10/',            # Máy tính
    'f11': '/f/dien-thoai.11/',          # Điện thoại
    'f249': '/f/mot-goc-rieng.249/',      # Một góc riêng
    'f594': '/f/kinh-te-tai-chinh.594/',  # Kinh tế tài chính
    'f595': '/f/bat-dong-san.595/',      # Bất động sản
    'f174': '/f/oto-xe-may.174/',        # Ô tô xe máy
}
```

3.2. Mô hình Producer-Consumer với Thread Pool

Hệ thống hoạt động theo mô hình **Producer-Consumer**:

- **Main Thread (Producer)**: Quét các trang danh mục (F17, F33, F10...) để thu thập danh sách Thread URL. Đây là "scout" — chỉ lấy danh sách, không crawl nội dung.
- **Worker Threads (Consumers)**: Mặc định **20 workers** hoạt động song song. Mỗi worker nhận một thread diễn đàn, crawl toàn bộ các page trong thread đó, parse từng comment và trích xuất dữ liệu.



Crawler sử dụng `ThreadPoolExecutor` với mặc định **20 worker threads**:

```
# voz_crawler_1m.py (dòng 507-552)
with ThreadPoolExecutor(max_workers=self.num_workers) as executor:
    # Submit các thread diễn đàn cho workers
    futures = {}
    for thread in threads[:20]:
        future = executor.submit(self.crawl_thread, thread)
        futures[future] = thread

    # Thu thập kết quả khi hoàn thành
    for future in as_completed(futures):
        thread = futures[future]
        try:
            docs = future.result()
            for doc in docs:
                # Ghi document vào file output
                with self.write_lock:
                    f.write(json.dumps(output_doc, ensure_ascii=False) + '\n')
                    f.flush()
                total_docs += 1
                pbar.update(1)
        except Exception as e:
            self.logger.error(f"Error processing thread: {e}")
```

Tại sao chọn multi-threading thay vì async?

Python có 2 cách phổ biến để xử lý nhiều request HTTP song song:

- **Async (aiohttp + asyncio)**: Dùng cú pháp `await` để chờ response mà không blocking. Một event loop duy nhất xử lý hàng nghìn request đồng thời trên **1 thread**. Hiệu suất rất cao nhưng yêu cầu **tất cả** thư viện trong pipeline phải hỗ trợ async-native.
- **Multi-threading (ThreadPoolExecutor)**: Mỗi thread chạy code đồng bộ (blocking) bình thường, nhưng OS lập lịch cho nhiều thread chạy song song. Khi thread A đang chờ response từ server, thread B vẫn tiếp tục xử lý.

Voz Forum sử dụng **Cloudflare Protection** — mọi request HTTP thông thường (kể cả từ `aiohttp`) đều bị chặn, server trả về trang "Just a moment..." thay vì nội dung thật. Để bypass, nhóm sử dụng thư viện `cloudscraper` — thư viện này giả lập trình duyệt Chrome, tự động giải Cloudflare challenge (JavaScript challenge, cookie verification) rồi mới gửi request thật.

Tuy nhiên, `cloudscraper` được xây dựng trên nền thư viện `requests` — chỉ hoạt động **đồng bộ (blocking)**:

```
# ❌ KHÔNG thể kết hợp cloudscraper với async
async def fetch(url):
    response = await scraper.get(url)
    # Lỗi: scraper.get() trả về Response object trực tiếp,
    # không phải coroutine – không thể await

# ✅ cloudscraper chỉ chạy đồng bộ – thread bị block cho đến khi có response
def fetch(url):
    response = scraper.get(url) # Block 0.5-2 giây chờ server
    return response.text
```

Nếu chỉ chạy đồng bộ trên 1 thread, mỗi request mất ~1 giây → crawl 1 triệu documents sẽ mất **~11 ngày**. Giải pháp là dùng **ThreadPoolExecutor với 20 threads**: mỗi thread gọi `scraper.get()` blocking riêng, nhưng 20 threads chạy đồng thời → tốc độ tăng gấp ~20 lần.

```
Thread 1: |--request A--| |--request D--| |--request G--| ...
Thread 2: |--request B--| |--request E--| |--request H--| ...
Thread 3: |--request C--| |--request F--| |--request I--| ...
          ↑ 3 request chạy cùng lúc, không chờ nhau
```

Đây là sự đánh đổi hợp lý: ưu tiên **bypass Cloudflare** (bắt buộc để crawl được Voz) thay vì hiệu suất tối ưu của async. Trong thực tế, multi-threading với 20 workers đã đủ nhanh để hoàn thành 1 triệu documents.

3.3. Giải quyết Race Condition

Khi 20 threads chạy đồng thời, nhiều tài nguyên chia sẻ cần được bảo vệ. Chúng tôi sử dụng **3 Lock riêng biệt** cho 3 tài nguyên khác nhau:

```
# voz_crawler_1m.py (dòng 192-195)
self.write_lock = Lock()      # Bảo vệ file output JSONL
self.stats_lock = Lock()     # Bảo vệ object thống kê
self.checkpoint_lock = Lock() # Bảo vệ checkpoint state
```

Vấn đề: Nếu không dùng Lock, 20 threads ghi file JSONL đồng thời sẽ gây **data corruption** — các dòng JSON bị xen lẫn vào nhau, tạo ra file không đọc được.

Giải pháp - Lock khi ghi file:

```
# voz_crawler_1m.py (dòng 570-572)
with self.write_lock:
    f.write(json.dumps(output_doc, ensure_ascii=False) + '\n')
    f.flush() # Đảm bảo dữ liệu ghi xuống disk ngay lập tức
```

Lock khi cập nhật thống kê:

```
# voz_crawler_1m.py (dòng 560-565)
with self.stats_lock:
    self.stats.total_docs += 1
    self.stats.total_words += word_count
    self.stats.doc_lengths.append(word_count)
    self.stats.authors.add(doc.get('author', 'unknown'))
    self.stats.threads.add(doc.get('thread_id', ''))
```

Lock khi đánh dấu post đã crawl:

```
# voz_crawler_1m.py (dòng 438-440)
with self.checkpoint_lock:
    self.checkpoint.mark_post_crawled(post_id)
```

Tại sao dùng 3 Lock thay vì 1? Nếu dùng 1 Lock chung, khi thread A đang ghi file thì thread B không thể cập nhật thống kê — gây bottleneck. 3 Lock riêng giúp các thao tác trên tài nguyên khác nhau diễn ra song song.

Class `CrawlStatistics` cũng có Lock riêng bên trong:

```
# voz_crawler_1m.py (dòng 99, 111-124)
class CrawlStatistics:
    def __init__(self):
        self.lock = Lock()
        # ...

    def add_document(self, doc: dict, tokens: List[str]):
        with self.lock:
            self.total_docs += 1
            doc_len = len(tokens)
            self.total_words += doc_len
            self.doc_lengths.append(doc_len)
            self.authors.add(doc.get('author', 'unknown'))

            # Chỉ track word frequency cho 100k docs đầu (tiết kiệm RAM)
            if self.total_docs <= 100000:
                for token in tokens[:100]:
                    self.word_freq[token] += 1
```

3.4. Cơ chế Checkpoint/Resume — Tạm dừng và chạy tiếp

Crawl 1 triệu documents mất nhiều giờ. Trong thời gian đó, có thể xảy ra mất mạng, máy tắt nguồn, hoặc Cloudflare chặn. Cơ chế **Checkpoint/Resume** cho phép dừng bất cứ lúc nào rồi chạy tiếp mà không mất dữ liệu.

Class `CrawlCheckpoint` — Lưu trạng thái:

```
# voz_crawler_1m.py (dòng 34-90)
class CrawlCheckpoint:
    def __init__(self, checkpoint_path='data/crawl_checkpoint.pkl'):
        self.crawled_threads: Set[str] = set() # Tập các thread đã crawl
        self.crawled_posts: Set[str] = set() # Tập các post đã crawl
        self.total_docs = 0 # Số docs đã thu thập
        self.failed_urls: List[str] = [] # URL lỗi (retry sau)
        self.last_forum_page: Dict[str, int] = {} # Trang cuối mỗi forum

    def save(self):
        """Serialize toàn bộ state ra file pickle"""
        data = {
            'crawled_threads': self.crawled_threads,
            'crawled_posts': self.crawled_posts,
            'total_docs': self.total_docs,
            'failed_urls': self.failed_urls,
            'last_forum_page': self.last_forum_page,
            'save_time': datetime.now().isoformat()
        }
        with open(self.checkpoint_path, 'wb') as f:
```

```

        pickle.dump(data, f)

def load(self) -> bool:
    """Load state từ file pickle khi restart"""
    if os.path.exists(self.checkpoint_path):
        with open(self.checkpoint_path, 'rb') as f:
            data = pickle.load(f)
            self.crawled_threads = data.get('crawled_threads', set())
            self.crawled_posts = data.get('crawled_posts', set())
            self.total_docs = data.get('total_docs', 0)
            self.failed_urls = data.get('failed_urls', [])
            self.last_forum_page = data.get('last_forum_page', {})
            return True
    return False

```

Khi khởi động — Xác định Resume hay Fresh Start:

```

# voz_crawler_1m.py (dòng 477-483)
if resume and self.checkpoint.load():
    self.logger.info(f"Resumed from checkpoint: {self.checkpoint.total_docs} docs")
    mode = 'a' # Append mode – KHÔNG ghi đè dữ liệu cũ
else:
    self.logger.info("Starting fresh crawl")
    mode = 'w' # Write mode – bắt đầu file mới

```

Auto-save checkpoint mỗi 60 giây:

```

# voz_crawler_1m.py (dòng 78-80, 583-589)
def should_save(self, interval_seconds: int = 60) -> bool:
    return (datetime.now() - self.last_save_time).seconds >= interval_seconds

# Trong vòng lặp crawl chính:
if self.checkpoint.should_save(self.checkpoint_interval):
    with self.checkpoint_lock:
        self.checkpoint.total_docs = total_docs
        self.checkpoint.last_forum_page[forum_id] = page
        self.checkpoint.save()
    self.logger.info(f"Checkpoint saved: {total_docs:,} docs")

```

Skip thread/post đã crawl (tránh trùng lặp khi resume):

```

# voz_crawler_1m.py (dòng 333-335) – Skip thread đã crawl
if self.checkpoint.is_thread_crawled(thread_id):
    continue

# voz_crawler_1m.py (dòng 404-408) – Skip post đã crawl
if self.checkpoint.is_post_crawled(post_id):
    with self.stats_lock:

```



```
self.stats.duplicates_skipped += 1
return None
```

Ví dụ thực tế: Crawler chạy được 500,000 docs thì mất mạng → Khởi động lại → Load checkpoint, file output mở ở chế độ append, bắt đầu từ thread/page tiếp theo → Tiếp tục thu thập đến 1,000,000 mà không crawl lại 500,000 docs cũ.

3.5. Xử lý lỗi mạng và Rate Limiting

```
# voz_crawler_1m.py (dòng 243-279)
def _get_page(self, scraper, url: str, max_retries: int = 3) -> Optional[str]:
    for attempt in range(max_retries):
        try:
            response = scraper.get(url, timeout=30)

            # Phát hiện Cloudflare challenge
            if "Just a moment" in response.text:
                self.logger.warning(f"Cloudflare challenge on {url}")
                time.sleep(5)
                continue

            if response.status_code == 200:
                return response.text

            # HTTP 429 – Server yêu cầu giảm tốc
            if response.status_code == 429:
                self.logger.warning("Rate limited, sleeping 30s")
                time.sleep(30)
                continue

            time.sleep(2) # Backoff cho các lỗi khác

        except Exception as e:
            self.logger.error(f"Error fetching {url}: {e}")
            time.sleep(3)

    # Sau 3 lần thất bại → lưu URL để retry sau
    with self.checkpoint_lock:
        self.checkpoint.failed_urls.append(url)
    return None
```

3.6. Anti-Detection với Cloudscraper và Connection Pooling

Voz Forum sử dụng Cloudflare Protection. Thư viện `cloudscraper` giả lập browser thật để bypass:

```
# voz_crawler_1m.py (dòng 218-236)
def _create_scraper(self):
    scraper = cloudscraper.create_scraper(
        browser={
            'browser': 'chrome',
            'platform': 'windows',
            'desktop': True
        }
    )
    # Connection pooling – tái sử dụng TCP connection
    adapter = requests.adapters.HTTPAdapter(
        pool_connections=10,
        pool_maxsize=20,
        max_retries=3
    )
    scraper.mount('https://', adapter)
    scraper.mount('http://', adapter)
    return scraper
```

Thêm random delay giữa các request để tránh bị detect:

```
# voz_crawler_1m.py (dòng 238-241)
def _random_delay(self):
    delay = random.uniform(*self.delay_range) # 0.1 - 0.3 giây
    time.sleep(delay)
```

3.7. Phân tích cú pháp HTML (Parsing)

Mỗi post Voz được parse chi tiết, bao gồm xử lý quote (trích dẫn):

```
# voz_crawler_1m.py (dòng 399-454)
def _parse_post(self, post_elem, thread_id, thread_title, thread_url):
    post_id = post_elem.get('data-content', '').replace('post-', '')

    # Skip post đã crawl
    if self.checkpoint.is_post_crawled(post_id):
        return None

    content_elem = post_elem.select_one('div.bbWrapper')

    # Tách quote ra khỏi nội dung chính
    quote_elem = content_elem.select_one('blockquote.bbCodeBlock--quote')
    if quote_elem:
        has_quote = True
        quote_title = quote_elem.select_one('div.bbCodeBlock-title')
        quoted_author = quote_title.get_text(strip=True)
        quote_elem.decompose() # Xóa quote khỏi DOM

    content_raw = content_elem.get_text(separator=' ', strip=True)
```

```

author = post_elem.select_one('.message-name .username').get_text(strip=True)
timestamp = post_elem.select_one('.message-attribution-main time').get('datetime')

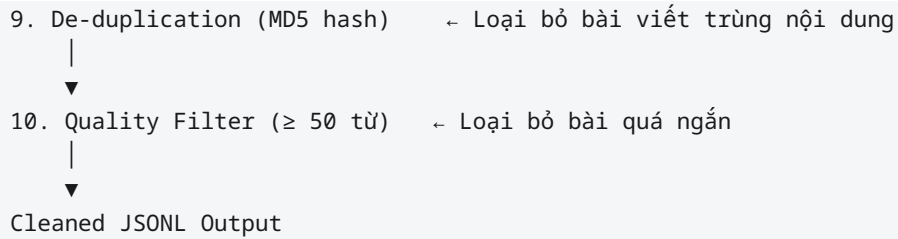
return {
    'doc_id': f"voz_t{thread_id}_p{post_id}",
    'thread_id': f"t{thread_id}",
    'thread_title': thread_title,
    'content': content_raw,
    'author': author,
    'timestamp': timestamp,
    'has_quote': has_quote,
    'quoted_author': quoted_author,
    'source': 'voz',
    'url': f"{self.BASE_URL}/p/{post_id}/"
}

```

4. Data Cleaning — Làm sạch dữ liệu

4.1. Pipeline tổng thể





4.2. Chuẩn hóa Unicode (Unicode Normalization)

Tiếng Việt trên máy tính có thể được lưu dưới 2 dạng khác nhau cho cùng một ký tự:

- **Dạng tổ hợp (Decomposed):** Ký tự `á` được lưu thành 2 byte: `a` + `dấu sắc`
- **Dạng dựng sẵn (Precomposed):** Ký tự `á` là 1 byte duy nhất

Nếu không chuẩn hóa, cùng từ "các" có thể tồn tại dưới 2 dạng mà máy tính coi là khác nhau → ảnh hưởng đến de-duplication và search.

```
# voz_crawler_1m.py (dòng 291-294) – Trong crawler, chuẩn hóa NFC ngay khi crawl
import unicodedata
text = unicodedata.normalize('NFC', text)
# NFC = Normalization Form Composed: chuyển tất cả về dạng dựng sẵn
# "a" + "dấu sắc" → "á" (1 ký tự duy nhất)
```

4.3. Loại bỏ HTML và URL

```
# parser.py (dòng 213-261)
def remove_html_tags(text: str) -> str:
    # Xóa script và style (cả nội dung bên trong)
    text = re.sub(r'<script[^>]*>.*?</script>', '', text, flags=re.DOTALL | re.IGNORECASE)
    text = re.sub(r'<style[^>]*>.*?</style>', '', text, flags=re.DOTALL | re.IGNORECASE)

    # Xóa HTML comments
    text = re.sub(r'<!--.*?-->', '', text, flags=re.DOTALL)

    # Xóa tất cả HTML tags
    text = re.sub(r'<[^>]+>', '', text)

    # Decode HTML entities
    html_entities = {
        '&nbsp;': ' ', '&amp;': '&', '&lt;': '<', '&gt;': '>',
        '&quot;': '"', '&#39;': "'", '&hellip;': '...', '&mdash;': '—',
    }
    for entity, char in html_entities.items():
        text = text.replace(entity, char)
```

```
return text
```

```
def remove_urls(text: str) -> str:
    text = re.sub(r'https?:\/\/[^\s<>"}|\\^`\\[\]]+', '', text)
    text = re.sub(r'www\.[^\s<>"}|\\^`\\[\]]+', '', text)
    return text
```

4.4. Xử lý Teencode — Thách thức đặc thù Social Listening

Dữ liệu diễn đàn Việt Nam chứa rất nhiều teencode (viết tắt, tiếng lóng). Đây là thách thức lớn nhất của chủ đề Social Listening so với các chủ đề khác.

Bộ từ điển teencode (80+ mục):

```
# parser.py (dòng 41-185)
TEENCODE_DICT = {
    # === TEENCODE PHỔ BIẾN ===
    'ko': 'không',      'dc': 'được',      'đc': 'được',
    'mk': 'mình',       'mik': 'mình',     'rep': 'trả lời',

    # === ĐẠI TỪ ===
    'ae': 'anh em',     'vk': 'vợ',       'gf': 'bạn gái',
    'bf': 'bạn trai',   'ny': 'người yêu', 'ngta': 'người ta',

    # === HÀNH ĐỘNG ===
    'bt': 'biết',       'bth': 'bình thường', 'sd': 'sử dụng',
    'trc': 'trước',     'cx': 'cũng',     'vs': 'với',

    # === VIẾT TẮT CHÍNH THỨC ===
    'bhyt': 'bảo hiểm y tế',      'bhxh': 'bảo hiểm xã hội',
    'bhtn': 'bảo hiểm thất nghiệp', 'thpt': 'trung học phổ thông',
    'cccd': 'căn cước công dân',   'cmnd': 'chứng minh nhân dân',
    'cntt': 'công nghệ thông tin',  'bds': 'bất động sản',

    # === VOZ-SPECIFIC SLANG ===
    'thím': 'bạn',      'vozer': 'thành viên voz',
    'thớt': 'topic',    'tks': 'cảm ơn',
}
```

Thuật toán thay thế — Boundary-Aware Regex:

Thách thức lớn nhất là **không phá hỏng các từ khác**. Ví dụ: "ko" phải thay thành "không" nhưng "không" không được bị thay thêm lần nữa, "iko" không được bị thay thành "ikkhông".

```
# parser.py (dòng 282-318)
def normalize_teencode(text: str) -> str:
```

```

result = text

# Ký tự ranh giới – KHÔNG thay thế nếu teencode nằm trong URL/hashtag
url_chars = r'a-zA-Zàáâãäåæçèéêëìíîïð... ' # Ký tự tiếng Việt
special_chars = r'/:@#\\-_' # Ký tự URL/hashtag

# Sort by length (dài trước) để tránh partial match
# Ví dụ: "bth" → "bình thường" phải xử lý TRƯỚC "bt" → "biết"
sorted_teencode = sorted(TEENCODE_DICT.items(),
                          key=lambda x: len(x[0]), reverse=True)

for teen, standard in sorted_teencode:
    # Lookbehind: KHÔNG preceded by chữ cái/số
    # Lookahead: KHÔNG followed by chữ cái/số
    pattern = (r'(?<[' + boundary_chars + r'0-9])' +
               re.escape(teen) +
               r'(?![ ' + boundary_chars + r'0-9])')

    result = re.sub(pattern, standard, result, flags=re.IGNORECASE)

return result

```

Tại sao loại bỏ ký tự đơn ambiguous?

```

# parser.py (dòng 176-184) – Các ký tự KHÔNG thay thế
# 'k': 'không',    # "1k" (1000), "OK", tên riêng chứa K
# 'e': 'em',       # "e = mc²", scientific notation
# 'r': 'rồi',      # "R" trong đơn vị, biến số
# 'v': 'vậy',     # "V" (voltage), tên riêng

```

Minh họa kết quả trên dữ liệu thật:

Input (raw)	Output (cleaned)
bhtn làm đc online ko nhĩ	bảo hiểm thất nghiệp làm được online không nhĩ
ae có ny chưa	anh em có người yêu chưa
trc e bt bth thôi	trước e biết bình thường thôi
thím nào ở sg cho ib	bạn nào ở sài gòn cho inbox

4.5. Tách từ tiếng Việt (Word Segmentation)

Tiếng Việt có nhiều **từ ghép** mà nếu tách riêng sẽ mất nghĩa. Thư viện `underthesea` giải quyết vấn đề này:

```
# parser.py (dòng 354-372)
def segment_vietnamese(text: str) -> str:
    if not SEGMENTER_AVAILABLE:
        return text
    try:
        segmented = word_tokenize(text, format='text')
        return segmented
    except Exception:
        return text # Fallback: giữ nguyên nếu lỗi
```

Ví dụ tách từ:

Input	Output (segmented)
trung tâm việc làm	trung_tâm việc_làm
bảo hiểm xã hội	bảo_hiểm_xã_hội
dễ tính, chỉ dẫn nhiệt tình	dễ_tính , chỉ_dẫn nhiệt_tình
kinh tế hơi hỏ	kinh_tế hơi hỏ

Dấu gạch dưới (_) nối các thành phần của từ ghép, giúp các thuật toán search (BM25, SPIMI) ở Milestone 2 hiểu đúng đơn vị từ vựng.

4.6. De-duplication bằng Content Hash

```
# parser.py (dòng 385-392)
def compute_content_hash(content: str) -> str:
    # Chuẩn hóa trước khi hash: lowercase + gộp khoảng trắng
    normalized = ' '.join(content.lower().split())
    return hashlib.md5(normalized.encode('utf-8')).hexdigest()
```

Trong pipeline chính, mỗi document được kiểm tra hash trước khi thêm vào output:

```
# parser.py (dòng 420-435)
def process_document(doc, seen_hashes, min_word_count=50):
    content_hash = compute_content_hash(content_clean_original)
    if content_hash in seen_hashes:
        return None # Trùng lặp → bỏ qua
    seen_hashes.add(content_hash)

    cleaned_text = clean_text(content_clean_original, normalize_teen=True)

    word_count = count_words(cleaned_text)
    if word_count < min_word_count:
```

```

        return None # Quá ngắn → bỏ qua

    segmented_text = segment_vietnamese(cleaned_text)
    # ...

```

Kết quả: Loại bỏ 1,999 documents trùng lặp từ tổng 1,008,664 đầu vào (tỷ lệ trùng 0.2%).

4.7. Batch Writing — Tối ưu I/O

Thay vì ghi từng document, pipeline gom thành batch rồi ghi một lần:

```

# parser.py (dòng 513-554)
buffer = []
for doc in tqdm(read_jsonl(input_path), total=total_lines, desc="Cleaning data"):
    cleaned_doc = process_document(doc, seen_hashes, min_word_count)

    if cleaned_doc:
        buffer.append(json.dumps(cleaned_doc, ensure_ascii=False))

        # Ghi khi buffer đầy (mỗi 10,000 docs)
        if len(buffer) >= batch_size:
            out_file.write('\n'.join(buffer) + '\n')
            buffer = []

# Ghi phần còn lại
if buffer:
    out_file.write('\n'.join(buffer) + '\n')

```

4.8. Xử lý Code-switching (Văn bản hỗn hợp Việt-Anh)

Trên diễn đàn Voz, người dùng thường xuyên trộn lẫn tiếng Việt và tiếng Anh trong cùng một câu — hiện tượng gọi là **Code-switching**. Ví dụ:

"Con laptop này chạy game AAA mượt lắm tím, build PC thì budget tầm 20tr là ok"

Pipeline xử lý Code-switching theo 3 cấp độ:

Cấp 1 — Chuyển đổi slang tiếng Anh phổ biến sang tiếng Việt:

```

# parser.py (dòng 148-157) – Teencode dict bao gồm cả tiếng Anh thông dụng
TEENCODE_DICT = {
    # ...
    # VOZ-SPECIFIC SLANG
    'thanks': 'cảm ơn',

```



```
'sorry': 'xin lỗi',
'pls': 'làm ơn',
'plz': 'làm ơn',
# Tech terms
'gg': 'google',
'ytb': 'youtube',
}
```

Chỉ chuyển đổi những từ tiếng Anh **thông dụng trên Voz** mà có tương đương tiếng Việt rõ ràng. Các thuật ngữ chuyên ngành như "laptop", "game", "bitcoin", "GPU" được **giữ nguyên** vì chúng là từ khóa tìm kiếm quan trọng.

Cấp 2 — Lowercase normalization cho cả hai ngôn ngữ:

```
# parser.py (dòng 345-346)
# Step 6: Convert to lowercase (for case-insensitive search with BM25/SPIMI)
text = text.lower()
```

Thao tác `lower()` chuẩn hóa cả tiếng Việt lẫn tiếng Anh, đảm bảo "Laptop" = "laptop" = "LAPTOP" khi tìm kiếm.

Cấp 3 — Word segmentation nhận diện từ tiếng Anh:

```
# parser.py (dòng 354-369)
def segment_vietnamese(text: str) -> str:
    segmented = word_tokenize(text, format='text')
    return segmented
```

Thư viện `underthesea` khi gặp từ tiếng Anh (ký tự Latin không dấu) sẽ giữ nguyên thay vì cố tách, trong khi vẫn ghép đúng từ ghép tiếng Việt bằng dấu gạch dưới:

Input (mixed)	Output (segmented)
"mua laptop gaming giá rẻ"	"mua laptop gaming giá_rẻ"
"update firmware cho router wifi"	"update firmware cho router wifi"
"bất động sản ở quận 7"	"bất_động_sản ở quận 7"

Cách tiếp cận này **bảo toàn khả năng tìm kiếm** cho cả từ khóa tiếng Việt lẫn tiếng Anh — phù hợp với thực tế sử dụng ngôn ngữ trên diễn đàn công nghệ Việt Nam.

5. Cấu trúc dữ liệu lưu trữ

5.1. Format JSONL

Mỗi dòng trong file output là một JSON object hoàn chỉnh:

```
{
  "doc_id": "voz_t1200082_p40521182",
  "thread_id": "t1200082",
  "thread_title": "Làm bảo hiểm thất nghiệp online",
  "content": "Cho mình hỏi bhtn làm đc online ko nhĩ...",
  "author": "Kaiser2013",
  "timestamp": "2026-01-22T09:23:57+0700",
  "source": "voz",
  "url": "https://voz.vn/p/40521182/",
  "content_clean": "Cho mình hỏi bhtn làm đc online ko nhĩ...",
  "word_count": 65,
  "text_cleaned": "cho mình hỏi bảo hiểm thất nghiệp làm được online không nhĩ...",
  "text_segmented": "cho mình hỏi bảo_hiểm thất_nghiệp làm được online không nhĩ...",
  "word_count_clean": 68
}
```

5.2. Giải thích các trường

Trường	Mô tả
doc_id	ID duy nhất: voz_t{thread}_p{post}
content	Nội dung gốc từ crawler
content_clean	Nội dung đã loại HTML/URL (chưa normalize teencode)
text_cleaned	Nội dung đã normalize teencode + lowercase
text_segmented	Nội dung đã tách từ tiếng Việt (dùng cho SPIMI ở M2)
word_count_clean	Số từ sau cleaning
timestamp	Thời gian đăng bài (ISO 8601 + timezone)

6. Thống kê dữ liệu (Data Insights)

6.1. Thống kê tổng quan

Chỉ số	Giá trị
Tổng documents	1,000,083
Tổng tokens	82,466,288
Kích thước từ vựng	822,837 từ duy nhất
Unique threads	100,030
Unique authors	37,629
Thời gian crawl	03/2020 — 01/2026

6.2. Thống kê độ dài document

Chỉ số	Giá trị
Ngắn nhất	50 từ
Dài nhất	7,558 từ
Trung bình	101.1 từ

6.3. Phân bố độ dài

Khoảng	Số lượng	Tỷ lệ
50–99 từ	710,563	71.1%
100–199 từ	227,781	22.8%
200–499 từ	54,229	5.4%
500+ từ	7,510	0.8%

Phần lớn bài viết trên diễn đàn có độ dài vừa phải (50–100 từ), phản ánh đặc thù của comment/reply ngắn gọn. Các bài dài (500+ từ) thường là bài review, hướng dẫn, hoặc nội quy.

6.4. Top 10 tác giả tích cực nhất

#	Tác giả	Số bài
1	xuantruong	4,314
2	Kia Seltos	2,417
3	cruelpham	2,319
4	Bing AI	1,897
5	ProUsa@Usa	1,868
6	ArterNis	1,812
7	Hoa Do	1,590
8	dangkhoa27187	1,585
9	Phanh Blank 2	1,525
10	Ma Bư Mập	1,493

6.5. Top 20 từ xuất hiện nhiều nhất

#	Từ	Tần suất
1	thì	1,633,653
2	là	1,450,851
3	có	1,405,418
4	không	1,376,417
5	nó	884,046
6	cũng	843,832
7	được	780,465
8	mà	728,105
9	cho	649,967
10	đi	633,669
11	làm	632,785

#	Từ	Tần suất
12	cái	614,074
13	người	560,353
14	mình	545,456
15	rồi	545,304
16	và	539,504
17	còn	538,137
18	của	528,344
19	phải	522,426
20	tôi	522,039

Các từ xuất hiện nhiều nhất đều là **hư từ** (function words) và **đại từ** tiếng Việt, phù hợp với đặc trưng ngôn ngữ tự nhiên. Ở Milestone 2, các từ này sẽ được xử lý thông qua stopwords removal và IDF weighting trong BM25.

6.6. Kết quả Cleaning Pipeline

Giai đoạn	Số lượng
Input (raw)	1,008,664
Duplicates loại bỏ	–1,999 (0.2%)
Too short loại bỏ (< 50 từ)	–6,582 (0.7%)
Errors	0
Output (cleaned)	1,000,083
Retention rate	99.1%

7. Hướng dẫn chạy lại (Reproducibility)

7.1. Cài đặt môi trường

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

7.2. Crawl dữ liệu

```
# Crawl mới – 1 triệu documents, 20 threads
python src/crawler/voz_crawler_1m.py --target 1000000 --workers 20

# Resume sau khi bị gián đoạn (tự động)
python src/crawler/voz_crawler_1m.py

# Tùy chỉnh
python src/crawler/voz_crawler_1m.py --target 500000 --workers 10 --min-words 30
```

7.3. Cleaning dữ liệu

```
python src/crawler/parser.py --input data/voz_1m.jsonl --output data/cleaned/
```

7.4. Tạo thống kê

```
python src/cleaner/data_statistics.py \
  --input data/data_clean/voz_cleaned.jsonl \
  --output data/data_clean/statistics.json
```

8. Kết luận

8.1. Thách thức gặp phải và giải pháp

Thách thức	Giải pháp
Cloudflare Protection chặn request	<code>cloudscraper</code> giả lập browser Chrome
Rate Limiting (HTTP 429)	Exponential backoff + random delay
Mất mạng / crash giữa chừng	Checkpoint/Resume với pickle serialization
Race condition khi multi-thread	3 Lock riêng cho file/stats/checkpoint
Teencode ambiguous (k, e, r)	Loại bỏ ký tự đơn, chỉ xử lý từ 2+ ký tự
Teencode trong URL/hashtag	Boundary-aware regex với lookbehind/lookahead
Từ ghép tiếng Việt	underthesea word segmentation
Document trùng lặp	MD5 hash-based de-duplication

8.2. Đánh giá kết quả

- **Khối lượng:** 1,000,083 documents — vượt yêu cầu 1,000,000
- **Chất lượng:** Dữ liệu sạch, đã tách từ, không lỗi font
- **Kỹ thuật:** Multi-threading, Checkpoint/Resume, Rate Limiting
- **Thống kê:** Báo cáo đầy đủ vocabulary, distribution, top authors/words